

Surpriser_API_Endpoints_Guide

RESUMEN GENERAL EJECUTIVO

Este documento recopila toda la arquitectura de endpoints de la API de Surpriser, organizada por módulos funcionales. La plataforma ofrece un sistema completo de autenticación con registro, login, recuperación de contraseña y cierre de sesión mediante tokens Sanctum. Incluye un flujo de suscripción desde la landing page para captar usuarios interesados.

El módulo de usuarios permite consultar el perfil autenticado, actualizar datos personales y obtener un dashboard público con estadísticas, skills, sorpresas creadas y realizadas, reviews y nivel global.

El sistema de sorpresas constituye el núcleo de la plataforma: permite crear, actualizar, iniciar, entregar, completar, cancelar y eliminar sorpresas, con reglas estrictas según el rol (creador o genius) y el estado de la sorpresa. También ofrece listados por usuario y un endpoint general para obtener todas las sorpresas.

El módulo de disputas permite abrir, consultar y resolver conflictos entre creadores y genios, con validaciones de permisos, estados y motivos. El panel de administración incluye herramientas avanzadas para gestionar usuarios, sorpresas y disputas, con filtros, paginación y acciones administrativas como banear o cancelar sorpresas.

Finalmente, el sistema de skills y niveles proporciona métricas de progreso, nivel global, dashboards especializados y sugerencias de genios basadas en reputación, experiencia y rendimiento. La lógica interna de niveles clasifica a los genios en rangos como SPARK, FLAME, GENIE y SULTAN.

En conjunto, este documento constituye la documentación técnica completa de la API de Surpriser, lista para ser incluida en un PDF profesional y utilizada como referencia para desarrollo, QA, integración y auditoría.

ÍNDICE COMPLETO DEL DOCUMENTO

1. Registro, Login, Logout y Recuperación de Contraseña

- 1.1 Registro de Usuario (POST /api/register)
- 1.2 Inicio de Sesión (POST /api/login)
- 1.3 Solicitud de Recuperación de Contraseña (POST /api/forgot-password)
- 1.4 Restablecer Contraseña (POST /api/reset-password)
- 1.5 Cerrar Sesión (POST /api/logout)

2. Landing Page

- 2.1 Suscripción a la Landing (POST /api/landing/subscribe)

3. Usuarios

- 3.1 Obtener Usuario Autenticado (GET /api/me)
- 3.2 Actualizar Perfil de Usuario (PUT /api/user/profile)
- 3.3 Dashboard de Usuario (GET /api/users/{id}/dashboard)

4. Sorpresas

- 4.1 Crear una Sorpresa (POST /api/surprises)
- 4.2 Actualizar una Sorpresa (PUT /api/surprises/{id})
- 4.3 Listar Todas las Sorpresas (GET /api/surprises)
- 4.4 Obtener una Sorpresa por ID (GET /api/surprises/{id})
- 4.5 Eliminar una Sorpresa (DELETE /api/surprises/{id})
- 4.6 Iniciar una Sorpresa (POST /api/surprises/{id}/start)
- 4.7 Entregar una Sorpresa (POST /api/surprises/{id}/deliver)
- 4.8 Completar una Sorpresa (POST /api/surprises/{id}/complete)
- 4.9 Cancelar una Sorpresa (POST /api/surprises/{id}/cancel)
- 4.10 Listar Sorpresas Creadas por un Usuario (GET /api/users/{id}/surprises-created)
- 4.11 Listar Sorpresas Realizadas por un Genius (GET /api/users/{id}/surprises-genius)

5. Disputas

- 5.1. Abrir una disputa (POST /api/surprises/{id}/dispute)
- 5.2. Ver una disputa (GET /api/disputes/{id})
- 5.3. Listar mis disputas (GET /api/my/disputes)
- 5.4. Listar disputas donde un usuario es creador (GET /api/users/{id}/disputes/creator)
- 5.5. Listar disputas donde un usuario es genius (GET /api/users/{id}/disputes/genius)

6. Conversaciones y mensajes

- 6.1. Listar conversaciones del usuario (GET /api/conversations)
- 6.2. Crear u obtener una conversación (POST /api/conversations)
- 6.3. Eliminar una conversación (DELETE /api/conversations/{conversationId})
- 6.4. Listar mensajes de una conversación (GET /api/conversations/{conversationId}/messages)
- 6.5. Enviar un mensaje (POST /api/messages)
- 6.6. Enviar un mensaje desde una sorpresa (POST /api/surprises/{surpriseId}/messages)
- 6.7. Descargar archivo de un mensaje (GET /api/messages/file/{id})

7. Ofertas y negociación

- 7.1. Enviar una oferta a una sorpresa (POST /api/surprises/{surpriseId}/offers)
- 7.2. Listar ofertas de una sorpresa (GET /api/surprises/{surpriseId}/offers)
- 7.3. Enviar una contraoferta (POST /api/offers/{offerId}/counter)

7.4. Aceptar una oferta (POST /api/offers/{offerId}/accept)

8. Archivos

- 8.1. Subir un archivo a una sorpresa (POST /api/surprises/{id}/files)
- 8.2. Listar archivos de una sorpresa (GET /api/surprises/{id}/files)
- 8.3. Descargar un archivo de sorpresa (GET /api/files/{id}/download)

9. Notificaciones

- 9.1. Listar notificaciones de un usuario (GET /api/users/{id}/notifications)
- 9.2. Marcar una notificación como leída (POST /api/notifications/{id}/read)
- 9.3. Marcar todas las notificaciones como leídas (POST /api/users/{id}/notifications/read-all)

10. Reviews

- 10.1. Crear una review de una sorpresa (POST /api/surprises/{id}/review)
- 10.2. Listar reviews recibidas por un usuario (GET /api/users/{id}/reviews)
- 10.3. Obtener rating promedio de un usuario (GET /api/users/{id}/rating)

11. Skills

- 11.1. Listar todas las skills del sistema (GET /api/skills)
- 11.2. Listar skills propuestas y activas de un usuario (GET /api/users/{id}/skills)
- 11.3. Actualizar skills propuestas de un usuario (POST /api/users/{id}/proposed-skills)
- 11.4. Top 10 de genios por skill (GET /api/skills/{skillId}/top)
- 11.5. Ranking por sorpresas completadas en una skill (GET /api/skills/{skillId}/ranking)
- 11.6. Progreso de un usuario en una skill (GET /api/users/{userId}/skills/{skillId}/progress)
- 11.7. Progreso del usuario en todas sus skills (GET /api/users/{userId}/skills/progress)
- 11.8. Historial de sorpresas completadas por skill (GET /api/users/{userId}/skills/{skillId}/history)
- 11.9. Nivel global de un usuario como genius (GET /api/users/{userId}/level)
- 11.10. Dashboard de skills del usuario (GET /api/users/{userId}/skills/dashboard)
- 11.11. Sugerencias de genios para una skill (GET /api/skills/{skillId}/genius-suggestions)

12. Admin

- 12.1. Dashboard general de administración (GET /api/admin/dashboard)
- 12.2. Listar disputas con filtros avanzados (GET /api/admin/disputes)
- 12.3. Resolver una disputa (POST /api/admin/disputes/{id}/resolve)
- 12.4. Listar usuarios del sistema (GET /api/admin/users)
- 12.5. Banear a un usuario (POST /api/admin/users/{id}/ban)
- 12.6. Desbanear a un usuario (POST /api/admin/users/{id}/unban)
- 12.7. Listar todas las sorpresas del sistema (GET /api/admin/surprises)
- 12.8. Cancelación forzada de una sorpresa (POST /api/admin/surprises/{id}/force-cancel)

1. REGISTRO, LOGIN, LOGOUT Y RECUPERACION DE CONTRASEÑA

1.1 Registro de Usuario (POST /api/register)

El endpoint POST /api/register permite registrar un nuevo usuario enviando los datos básicos necesarios para crear la cuenta. El backend valida el nombre, el email y la contraseña, sanitiza únicamente los campos name y email, crea el usuario en la base de datos y envía automáticamente un correo de verificación. La respuesta indica que el usuario debe revisar su email para activar la cuenta. Este endpoint está limitado a 3 solicitudes por minuto.

Datos que debe enviar el cliente (JSON escrito en texto):

```
{
  "name": "Nombre del usuario",
  "email": "correo@example.com",
  "password": "contraseña123",
  "password_confirmation": "contraseña123"
}
```

Respuesta típica de la API (texto tipo JSON):

```
{
  "message": "Usuario registrado. Revisa tu email para verificar la cuenta."
}
```

1.2 Inicio de Sesión (POST /api/login)

El endpoint POST /api/login permite que un usuario inicie sesión enviando su email y contraseña. El backend valida que ambos campos estén presentes y que el email tenga un formato válido. Antes de procesar el login, el sistema aplica un control de intentos mediante RateLimiter: si un usuario falla demasiadas veces, la API devuelve un error indicando que debe esperar antes de volver a intentarlo. El email se sanitiza para evitar contenido malicioso, mientras que la contraseña nunca se sanitiza para no alterar su valor. El sistema comprueba que el usuario exista, que haya verificado su email previamente y que la contraseña sea correcta. Si todo es válido, se genera un token de acceso (Sanctum) que el cliente debe usar en las siguientes peticiones autenticadas. Si algo falla, la API devuelve un mensaje de error indicando el motivo. Este endpoint tiene un límite de 5 solicitudes por minuto.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "email": "correo@example.com",
  "password": "contraseña123"
}
```

Posibles respuestas de la API (texto tipo JSON):

Éxito:

```
{
  "token": "token_de_autenticacion_generado_por_sanctum"
}
```

Email no verificado:

```
{  
"error": "Debes verificar tu email antes de iniciar sesión"  
}
```

Credenciales incorrectas:

```
{  
"error": "Credenciales inválidas"  
}
```

Demasiados intentos:

```
{  
"error": "Demasiados intentos. Intenta más tarde."  
}
```

1.3 Solicitud de Recuperación de Contraseña (POST /api/forgot-password)

El endpoint POST /api/forgot-password permite que un usuario solicite un enlace para restablecer su contraseña. El cliente debe enviar únicamente el email asociado a la cuenta. El backend valida que el email tenga un formato correcto y, si existe en la base de datos, intenta enviar el correo con el enlace de recuperación. Si el envío falla, la API responde con un mensaje de error. Este endpoint no requiere autenticación y está protegido con un límite de solicitudes para evitar abusos.

Datos que debe enviar el cliente (JSON en texto):

```
{  
"email": "correo@example.com"  
}
```

Respuesta típica de la API en caso de éxito:

```
{  
"message": "Email enviado"  
}
```

Respuesta típica si el email no existe o no se pudo enviar:

```
{  
"error": "No se pudo enviar el email"  
}
```

Respuesta típica cuando Laravel detecta que el usuario no existe:

```
{  
"error": "passwords.user"  
}
```

1.4 Restablecer Contraseña (POST /api/reset-password)

El endpoint POST /api/reset-password permite completar el proceso de recuperación de contraseña. El cliente debe enviar el token recibido por email, junto con el email del usuario, la nueva contraseña y su confirmación. El backend valida que todos los campos

sean correctos, verifica que el token sea válido y que coincida con el email, y si todo es correcto actualiza la contraseña del usuario en la base de datos. La contraseña se encripta antes de guardarse. Si el token es incorrecto, expiró o no coincide, la API devuelve un error. Este endpoint no requiere autenticación.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "token": "token_recibido_por_email",
  "email": "correo@example.com",
  "password": "nuevaContraseña123",
  "password_confirmation": "nuevaContraseña123"
}
```

Respuesta típica de la API en caso de éxito:

```
{
  "message": "Contraseña actualizada"
}
```

Respuesta típica si el token es inválido o expiró:

```
{
  "error": "Token inválido"
}
```

1.5 Cerrar Sesión (POST /api/logout)

El endpoint POST /api/logout permite cerrar la sesión del usuario autenticado. Para poder usarlo, el cliente debe enviar el token de autenticación (Sanctum) en el encabezado Authorization como “Bearer token”. El backend identifica al usuario mediante ese token y elimina todos los tokens asociados a su cuenta, cerrando la sesión en todos los dispositivos o sesiones activas. Una vez eliminados los tokens, la API devuelve un mensaje indicando que la sesión ha sido cerrada correctamente. Este endpoint requiere autenticación previa.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo JSON. Solo necesita el token en el header Authorization.)

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta típica de la API (texto tipo JSON):

```
{
  "message": "Sesión cerrada"
}
```

2. LANDING PAGE

2.1 Suscripción a la Landing (POST /api/landing/subscribe)

El endpoint POST /api/landing/subscribe permite registrar el email de un usuario interesado en Surpriser desde la landing page. El cliente envía únicamente un email. El backend valida que el email tenga formato correcto y que no exista previamente en la tabla landing_subscribers. Si el email ya existe, la validación lanza una excepción, pero el sistema la ignora para evitar mostrar errores al usuario y permitir que la experiencia sea fluida. Si el email es nuevo, se guarda en la base de datos junto con las marcas de tiempo created_at y updated_at. Tras procesar la solicitud, la API redirige siempre al usuario a la página <https://surpriser.app/gracias.html>, independientemente de si el email era nuevo o ya estaba registrado. Este endpoint no requiere autenticación.

Datos que debe enviar el cliente (JSON en texto):

```
{  
  "email": "correo@example.com"  
}
```

Comportamiento de la API:

- Si el email es válido y no existe: se guarda y se redirige a la página de gracias.
- Si el email ya existe: no se guarda nada nuevo y se redirige igualmente a la página de gracias.
- Si el email no tiene formato válido: se produce un error de validación (solo visible si se llama desde un cliente tipo Postman; en la landing no ocurre porque el formulario envía datos correctos).

Respuesta final (redirección):

Redirige a: <https://surpriser.app/gracias.html>

3. USUARIOS

3.1 Obtener Usuario Autenticado (GET /api/me)

El endpoint GET /api/me devuelve la información del usuario actualmente autenticado. Para usarlo, el cliente debe enviar el token de autenticación (Sanctum) en el encabezado Authorization como "Bearer token". El backend identifica al usuario mediante ese token y responde con todos los datos del usuario almacenados en la base de datos (id, nombre, email, timestamps, etc.). Este endpoint es útil para obtener el perfil del usuario después de iniciar sesión o para verificar si el token sigue siendo válido. Requiere autenticación previa.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo necesita el token en el header Authorization.)

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta típica de la API (texto tipo JSON):

```
{
  "id": 1,
  "name": "Nombre del usuario",
  "email": "correo@example.com",
  "email_verified_at": "2026-05-01T10:00:00.000000Z",
  "created_at": "2026-04-20T12:00:00.000000Z",
  "updated_at": "2026-04-20T12:00:00.000000Z"
}
```

(Los campos pueden variar según tu modelo User.)

3.2 Actualizar Perfil de Usuario (PUT /api/user/profile)

El endpoint PUT /api/user/profile permite que el usuario autenticado actualice los datos de su perfil. El cliente puede enviar cualquiera de los campos opcionales: name, username, bio, phone, location_city, location_country y avatar. Todos los campos son validados según su tipo y longitud, y el username debe ser único excepto para el propio usuario. El backend sanitiza únicamente los campos de texto libre para evitar contenido malicioso, sin modificar datos sensibles como contraseñas. Antes de actualizar, el sistema guarda los valores antiguos del usuario para auditoría y luego registra los cambios mediante AuditService. Finalmente, devuelve un mensaje de éxito junto con los datos actualizados del usuario. Este endpoint requiere autenticación mediante token.

Datos que puede enviar el cliente (JSON en texto):

```
{
  "name": "Nuevo nombre",
  "username": "nuevo_username",
  "bio": "Descripción del usuario",
  "phone": "+34 600000000",
  "location_city": "Barcelona",
  "location_country": "España",
  "avatar": "https://url-de-avatar.com/imagen.jpg"
}
```

```
}
```

(Todos los campos son opcionales. El cliente puede enviar solo los que quiera actualizar.)

Respuesta típica de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": 1,
    "name": "Nuevo nombre",
    "username": "nuevo_username",
    "bio": "Descripción del usuario",
    "phone": "+34 600000000",
    "location_city": "Barcelona",
    "location_country": "España",
    "avatar": "https://url-de-avatar.com/imagen.jpg",
    "...otros_campos_del_usuario"
  }
}
```

3.3 Dashboard de Usuario (GET /api/users/{id}/dashboard)

El endpoint GET /api/users/{id}/dashboard devuelve un resumen completo del perfil público del usuario, incluyendo sus habilidades, estadísticas de sorpresas creadas y realizadas, últimas reviews recibidas y un nivel global calculado a partir de sus skills. Este endpoint está pensado para mostrar un panel de información consolidada tanto para el propio usuario como para otros usuarios que visiten su perfil. No requiere autenticación obligatoria (según el código actual), ya que solo consulta información pública del usuario.

El backend realiza varias operaciones:

1. Obtiene los datos del usuario.
2. Recupera todas sus skills con nivel, progreso y XP total.
3. Obtiene las últimas 5 sorpresas creadas por el usuario y estadísticas de estado (total, open, in_progress, delivered, completed).
4. Obtiene las últimas 5 sorpresas realizadas como genius y sus estadísticas.
5. Recupera las últimas 5 reviews recibidas y calcula la media de rating.
6. Calcula un nivel global basado en la media de niveles de skills.
7. Devuelve toda la información agrupada en un único JSON estructurado.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:

/api/users/15/dashboard

Respuesta típica de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
```

```
"user": { ...datos del usuario... },
"skills": [
{
"skill_id": 1,
"skill_name": "Fotografía",
"level": 3,
"progress": 45,
"xp_total": 1200
},
{
"skill_id": 2,
"skill_name": "Diseño",
"level": 2,
"progress": 10,
"xp_total": 300
}
],
"created_surprises": {
"latest": [ ...últimas 5 sorpresas creadas... ],
"stats": {
"total": 12,
"open": 3,
"in_progress": 2,
"delivered": 4,
"completed": 3
}
},
"done_surprises": {
"latest": [ ...últimas 5 sorpresas realizadas como genius... ],
"stats": {
"total": 20,
"in_progress": 1,
"delivered": 7,
"completed": 12
}
},
"reviews": {
"latest": [ ...últimas 5 reviews... ],
"rating": 4.75
},
"global": {
"global_level": 2.5,
"total_skills": 7,
"total_reviews": 18,
"total_surprises": 32
}
}
}
```

4. SORPRESAS

4.1 Crear una Sorpresa (POST /api/surprises)

El endpoint POST /api/surprises permite crear una nueva sorpresa dentro de la plataforma. El cliente debe enviar los datos básicos de la sorpresa: el usuario creador, un genius opcional, el título, la descripción, el precio, la fecha límite, la skill asociada, el tamaño y si es urgente. El backend valida todos los campos, sanitiza únicamente el título y la descripción para evitar contenido malicioso, y fuerza que el estado inicial de la sorpresa sea siempre “open”, ignorando cualquier valor enviado por el cliente. Si el creador y el genius son el mismo usuario, la API devuelve un error. Si todo es correcto, se crea la sorpresa, se registra una auditoría y se dispara un evento de notificación. Finalmente, la API devuelve la sorpresa recién creada junto con un mensaje de éxito.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "creator_id": 15,
  "genius_id": 22,
  "title": "Título de la sorpresa",
  "description": "Descripción detallada de la sorpresa",
  "status": "open",
  "price": 50,
  "deadline": "2026-06-01",
  "skill_id": 3,
  "size": "MEDIUM",
  "is_urgent": true
}
```

(El campo “status” siempre será reemplazado por “open” internamente.)

(El campo “genius_id” puede ser null.)

Respuesta típica de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Surprise created successfully",
  "data": {
    "id": 120,
    "creator_id": 15,
    "genius_id": 22,
    "title": "Título de la sorpresa",
    "description": "Descripción detallada de la sorpresa",
    "status": "open",
    "price": 50,
    "deadline": "2026-06-01",
    "skill_id": 3,
    "size": "MEDIUM",
    "is_urgent": true,
    "created_at": "2026-05-05T12:00:00.000000Z",
    "updated_at": "2026-05-05T12:00:00.000000Z"
  }
}
```

Respuesta de error si creator_id y genius_id son iguales:

```
{  
"error": "Creator and genius cannot be the same user"  
}
```

4.2 Actualizar una Sorpresa (PUT /api/surprises/{id})

El endpoint PUT /api/surprises/{id} permite modificar una sorpresa existente, pero con reglas estrictas. Solo el creador puede modificarla y únicamente mientras la sorpresa esté en estado open o in_progress. El genius nunca puede modificar una sorpresa. Además, si la sorpresa está en estado delivered o completed, ya no puede cambiarse. El backend aplica validaciones distintas según el estado actual de la sorpresa: en estado open se pueden modificar casi todos los campos; en estado in_progress solo se permite actualizar la descripción y extender la fecha límite. El sistema sanitiza únicamente los campos de texto libre (título, descripción y datos del destinatario). Nunca permite cambiar manualmente el estado ni asignar un genius. Antes de actualizar, se guardan los valores antiguos para auditoría y luego se registran los cambios. Si se modifica el deadline y la sorpresa tiene genius asignado, se envía una notificación. Finalmente, la API devuelve la sorpresa actualizada.

Datos que puede enviar el cliente (JSON en texto):

(Los campos permitidos dependen del estado actual de la sorpresa.)

Si la sorpresa está en estado “open”:

```
{  
"title": "Nuevo título",  
"description": "Nueva descripción",  
"price": 60,  
"deadline": "2026-06-10",  
"size": "LARGE",  
"is_urgent": false,  
"target_name": "Nombre del destinatario",  
"target_city": "Barcelona",  
"target_country": "España",  
"target_lat": 41.390205,  
"target_lng": 2.154007  
}
```

Si la sorpresa está en estado “in_progress”:

```
{  
"description": "Nueva descripción",  
"deadline": "2026-06-10"  
}
```

Campos que NUNCA se pueden enviar:

status
genius_id

Respuestas típicas de la API (texto tipo JSON):

Éxito:

```
{
  "success": true,
  "message": "Surprise updated successfully",
  "data": { ...sorpresa_actualizada... }
}
```

El genius intenta modificar:

```
{
  "error": "Genius cannot modify surprise details"
}
```

La sorpresa está completed:

```
{
  "error": "Completed surprises cannot be modified"
}
```

La sorpresa está delivered:

```
{
  "error": "Delivered surprises cannot be modified"
}
```

El usuario no es el creador:

```
{
  "error": "Only the creator can modify the surprise"
}
```

4.3 Listar Todas las Sorpresas (GET /api/surprises)

El endpoint GET /api/surprises devuelve la lista completa de sorpresas registradas en la plataforma. Cada sorpresa incluye sus relaciones cargadas: el usuario creador, el genius asignado (si existe), los archivos asociados y las reviews. Este endpoint está pensado para obtener una vista global de todas las sorpresas, normalmente para paneles administrativos o listados generales. No requiere cuerpo en la petición y, según el código actual, tampoco requiere autenticación.

El backend obtiene todas las sorpresas mediante una consulta con relaciones (creator, genius, files, reviews) y devuelve los resultados en un JSON estructurado.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo.)

Respuesta típica de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 120,
      "creator_id": 15,
      "genius_id": 22,
      "title": "Título de la sorpresa",

```

```

"description": "Descripción",
"status": "open",
"price": 50,
"deadline": "2026-06-01",
"skill_id": 3,
"size": "MEDIUM",
"is_urgent": false,
"created_at": "2026-05-05T12:00:00.000000Z",
"updated_at": "2026-05-05T12:00:00.000000Z",
"creator": { ...datos del creador... },
"genius": { ...datos del genius... },
"files": [ ...archivos asociados... ],
"reviews": [ ...reviews asociadas... ]
},
{ ...más sorpresas... }
]
}

```

4.4 Obtener una Sorpresa por ID (GET /api/surprises/{id})

El endpoint GET /api/surprises/{id} devuelve toda la información detallada de una sorpresa específica. El backend busca la sorpresa por su ID y carga automáticamente sus relaciones: el usuario creador, el genius asignado (si existe), los archivos asociados y las reviews. Si la sorpresa no existe, se devuelve un error 404. Este endpoint es útil para mostrar la ficha completa de una sorpresa en la aplicación. No requiere cuerpo en la petición y, según el código actual, tampoco requiere autenticación.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:
/ api / surprises / 120

Respuesta típica de la API (texto tipo JSON):

```

{
"success": true,
"data": {
"id": 120,
"creator_id": 15,
"genius_id": 22,
"title": "Título de la sorpresa",
"description": "Descripción detallada",
"status": "open",
"price": 50,
"deadline": "2026-06-01",
"skill_id": 3,
"size": "MEDIUM",
"is_urgent": false,
"created_at": "2026-05-05T12:00:00.000000Z",
"updated_at": "2026-05-05T12:00:00.000000Z",
"creator": { ...datos del creador... },

```

```
"genius": { ...datos del genius... },
"files": [ ...archivos asociados... ],
"reviews": [ ...reviews asociadas... ]
}
}
```

Respuesta si la sorpresa no existe:

```
{
"message": "No query results for model [App\\Models\\Surprise] 9999"
}
```

4.5 Eliminar una Sorpresa (DELETE /api/surprises/{id})

El endpoint DELETE /api/surprises/{id} elimina una sorpresa de la base de datos. El backend busca la sorpresa por su ID y, si existe, la elimina permanentemente. Si no existe, Laravel devuelve automáticamente un error 404. Este endpoint no incluye validaciones de permisos en el código mostrado, por lo que cualquier usuario autenticado que pueda acceder a esta ruta podría eliminar sorpresas; normalmente esto se restringiría mediante middleware o políticas, pero en el código actual no se aplica ninguna restricción adicional.

Después de eliminar la sorpresa, la API devuelve un mensaje indicando que la operación se realizó correctamente.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:
/ api / surprises / 120

Respuesta típica de la API (texto tipo JSON):

```
{
"success": true,
"message": "Surprise deleted successfully"
}
```

Respuesta si la sorpresa no existe:

```
{
"message": "No query results for model [App\\Models\\Surprise] 9999"
}
```

4.6 Iniciar una Sorpresa (POST /api/surprises/{id}/start)

El endpoint POST /api/surprises/{id}/start permite que el genius asignado inicie oficialmente una sorpresa. Solo el genius puede hacerlo y únicamente cuando la sorpresa está en estado open. Si otro usuario intenta iniciarla o si la sorpresa no está en el estado correcto, la API devuelve un error. Al iniciarse, el estado cambia a in_progress, se registra una auditoría y se envía una notificación.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / surprises / 120 / start

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Surprise started",
  "data": { ...sorpresa_actualizada... }
}
```

Si el usuario NO es el genius asignado:

```
{ "error": "Only the assigned genius can start the surprise" }
```

Si la sorpresa NO está en estado open:

```
{ "error": "Only open surprises can be started" }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

4.7 Entregar una Sorpresa (POST /api/surprises/{id}/deliver)

El endpoint POST /api/surprises/{id}/deliver permite que el genius asignado marque una sorpresa como entregada. Solo el genius puede hacerlo y únicamente cuando la sorpresa está en estado in_progress. Si otro usuario intenta entregarla o si la sorpresa no está en el estado correcto, la API devuelve un error. Al entregarse, el estado cambia a delivered, se registra una auditoría y se envía una notificación.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / surprises / 120 / deliver

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Surprise delivered",
  "data": { ...sorpresa_actualizada... }
}
```

Si el usuario NO es el genius asignado:

```
{ "error": "Only the assigned genius can deliver the surprise" }
```

Si la sorpresa NO está en estado in_progress:

```
{ "error": "Only surprises in progress can be delivered" }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

4.8 Completar una Sorpresa (POST /api/surprises/{id}/complete)

El endpoint POST /api/surprises/{id}/complete permite que el creador marque una sorpresa como completada. Solo el creador puede hacerlo y únicamente cuando la sorpresa está en estado delivered. Si otro usuario intenta completarla o si la sorpresa no está en el estado correcto, la API devuelve un error. Al completarse, el estado cambia a completed, se registra la fecha de finalización, se registra una auditoría, se activan las skills del genius, se añade experiencia y se envía una notificación.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / surprises / 120 / complete

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{  
  "success": true,  
  "message": "Surprise completed and skills updated"  
}
```

Si el usuario NO es el creador:

```
{ "error": "Only the creator can complete the surprise" }
```

Si la sorpresa NO está en estado delivered:

```
{ "error": "Only delivered surprises can be completed" }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

4.9 Cancelar una Sorpresa (POST /api/surprises/{id}/cancel)

El endpoint POST /api/surprises/{id}/cancel permite cancelar una sorpresa. Pueden cancelarla el creador, el genius asignado o un admin. Solo se pueden cancelar sorpresas en estado open o in_progress. El cliente debe enviar un motivo de cancelación (reason_key) y un texto opcional (reason_text). El backend valida quién cancela, valida el estado, valida el motivo, clasifica el tipo de motivo, registra la cancelación, cambia el estado a cancelled, aplica penalización si cancela el genius, registra auditoría y envía notificación.

Datos que debe enviar el cliente (JSON en texto):

```
{  
  "reason_key": "illness",
```

```
"reason_text": "Estoy enfermo y no puedo continuar"
}
```

Valores permitidos en reason_key:

illness, personal_issue, force_majeure, technical_issue, no_time, uncomfortable, cant_now, no_reason

Ejemplo de URL:

/ api / surprises / 120 / cancel

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
"success": true,
"message": "Surprise cancelled",
"reason_type": "valid|doubtful|invalid",
"data": { ...sorpresa_actualizada... }
}
```

Si el usuario NO es creador, genius o admin:

```
{ "error": "You cannot cancel this surprise" }
```

Si la sorpresa NO está en estado open o in_progress:

```
{ "error": "This surprise cannot be cancelled" }
```

Si falta reason_key o no es válido:

```
{ "message": "The reason_key field is required" }
```

o

```
{ "message": "The selected reason_key is invalid" }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

4.10 Listar Sorpresas Creadas por un Usuario (GET /api/users/{id}/surprises-created)

El endpoint GET /api/users/{id}/surprises-created devuelve todas las sorpresas creadas por un usuario específico. El backend filtra por creator_id y carga las relaciones files y reviews para cada sorpresa. No requiere cuerpo en la petición y, según el código actual, tampoco requiere autenticación. Este endpoint es útil para mostrar el historial de sorpresas creadas por un usuario en su perfil o dashboard.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:

/ api / users / 15 / surprises-created

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 120,
      "creator_id": 15,
      "genius_id": 22,
      "title": "Título de la sorpresa",
      "description": "Descripción",
      "status": "open",
      "price": 50,
      "deadline": "2026-06-01",
      "skill_id": 3,
      "size": "MEDIUM",
      "is_urgent": false,
      "created_at": "2026-05-05T12:00:00.000000Z",
      "updated_at": "2026-05-05T12:00:00.000000Z",
      "files": [ ...archivos... ],
      "reviews": [ ...reviews... ]
    },
    { ...más sorpresas creadas por el usuario... }
  ]
}
```

Si el usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

4.11 Listar Sorpresas Realizadas por un Genius (GET /api/users/{id}/surprises-genius)

El endpoint GET /api/users/{id}/surprises-genius devuelve todas las sorpresas en las que un usuario ha participado como genius. El backend filtra por genius_id y carga las relaciones files y reviews para cada sorpresa. No requiere cuerpo en la petición y, según el código actual, tampoco requiere autenticación. Este endpoint es útil para mostrar el historial de trabajos realizados por un genius en su perfil o dashboard.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:

/ api / users / 22 / surprises-genius

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 140,
      "creator_id": 15,
      "genius_id": 22,
      "title": "Sorpresa realizada",
```

```
"description": "Descripción",
"status": "completed",
"price": 80,
"deadline": "2026-05-20",
"skill_id": 4,
"size": "LARGE",
"is_urgent": false,
"created_at": "2026-05-01T12:00:00.000000Z",
"updated_at": "2026-05-02T12:00:00.000000Z",
"files": [ ...archivos... ],
"reviews": [ ...reviews... ]
},
{ ...más sorpresas realizadas por el genius... }
]
}
```

Si el usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

5. DISPUTAS

5.1 Abrir una Disputa (POST /api/surprises/{id}/dispute)

El endpoint POST /api/surprises/{id}/dispute permite abrir una disputa entre el creador y el genius de una sorpresa. Solo pueden abrirla quienes participaron en la sorpresa (creator o genius) y únicamente cuando la sorpresa está en estado delivered o completed. El cliente debe enviar un motivo (reason). El backend valida permisos, estado, motivo, y que no exista ya una disputa abierta. Si todo es correcto, crea la disputa y la devuelve.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "reason": "El trabajo no cumple con lo solicitado"
}
```

Ejemplo de URL:

/ api / surprises / 120 / dispute

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Dispute opened",
  "data": {
    "id": 55,
    "surprise_id": 120,
    "creator_id": 15,
    "genius_id": 22,
    "opened_by": 15,
    "reason": "El trabajo no cumple con lo solicitado",
    "status": "open",
    "created_at": "2026-05-05T12:00:00.000000Z",
    "updated_at": "2026-05-05T12:00:00.000000Z"
  }
}
```

Si el usuario NO es creator ni genius:

```
{ "error": "Not authorized" }
```

Si la sorpresa NO está en estado delivered o completed:

```
{ "error": "Dispute not allowed in this status" }
```

Si falta reason o es inválido:

```
{ "message": "The reason field is required" }
```

Si ya existe una disputa abierta:

```
{ "error": "Dispute already open" }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

5.2 Ver una Disputa (GET /api/disputes/{id})

El endpoint GET /api/disputes/{id} devuelve la información completa de una disputa específica. Solo pueden verla el creador o el genius involucrados en la sorpresa asociada. El admin no accede por este endpoint, sino por el panel de administración. El backend carga la disputa junto con sus relaciones: la sorpresa, el creador y el genius. Si el usuario no está autorizado o la disputa no existe, se devuelve un error.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / disputes / 55

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "id": 55,
    "surprise_id": 120,
    "creator_id": 15,
    "genius_id": 22,
    "opened_by": 15,
    "reason": "El trabajo no cumple con lo solicitado",
    "status": "open",
    "created_at": "2026-05-05T12:00:00.000000Z",
    "updated_at": "2026-05-05T12:00:00.000000Z",
    "surprise": { ...datos de la sorpresa... },
    "creator": { ...datos del creador... },
    "genius": { ...datos del genius... }
  }
}
```

Si el usuario NO es creador ni genius:

```
{ "error": "Not authorized" }
```

Si la disputa no existe:

```
{ "message": "No query results for model [App\\Models\\Dispute] 9999" }
```

5.3 Listar Mis Disputas (GET /api/my/disputes)

El endpoint GET /api/my/disputes devuelve todas las disputas en las que el usuario autenticado participa, ya sea como creador o como genius. El backend carga la disputa junto con la sorpresa asociada, ordena por fecha de creación descendente y devuelve los resultados paginados (20 por página). No requiere cuerpo en la petición; solo el token de autenticación.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo token en el header.)

Ejemplo de URL:
/api/my/disputes

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 55,
        "surprise_id": 120,
        "creator_id": 15,
        "genius_id": 22,
        "opened_by": 15,
        "reason": "El trabajo no cumple con lo solicitado",
        "status": "open",
        "created_at": "2026-05-05T12:00:00.000000Z",
        "updated_at": "2026-05-05T12:00:00.000000Z",
        "surprise": { ...datos de la sorpresa... }
      },
      { ...más disputas del usuario... }
    ],
    "last_page": 3,
    "total": 45
  }
}
```

Si el usuario no tiene disputas:

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [],
    "last_page": 1,
    "total": 0
  }
}
```

Si el usuario no está autenticado:
{ "message": "Unauthenticated." }

5.4 Listar Disputas Donde un Usuario es el Creador (GET /api/users/{id}/disputes/creator)

El endpoint GET /api/users/{id}/disputes/creator devuelve todas las disputas en las que un usuario participa como creador. El backend filtra por creator_id, carga las relaciones surprise y genius, ordena por fecha de creación descendente y devuelve los resultados paginados (20 por página). No requiere cuerpo en la petición.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:
/ api / users / 15 / disputes / creator

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 55,
        "surprise_id": 120,
        "creator_id": 15,
        "genius_id": 22,
        "opened_by": 15,
        "reason": "El trabajo no cumple con lo solicitado",
        "status": "open",
        "created_at": "2026-05-05T12:00:00.000000Z",
        "updated_at": "2026-05-05T12:00:00.000000Z",
        "surprise": { ...datos de la sorpresa... },
        "genius": { ...datos del genius... }
      },
      { ...más disputas donde el usuario es creador... }
    ],
    "last_page": 2,
    "total": 25
  }
}
```

Si el usuario no existe:
{ "message": "No query results for model [App\\Models\\User] 9999" }

5.5 Listar Disputas Donde un Usuario es el Genius (GET /api/users/{id}/disputes/genius)

El endpoint GET /api/users/{id}/disputes/genius devuelve todas las disputas en las que un usuario participa como genius. El backend filtra por genius_id, carga las relaciones surprise y creator, ordena por fecha de creación descendente y devuelve los resultados paginados (20 por página). No requiere cuerpo en la petición.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL.)

Ejemplo de URL:

/ api / users / 22 / disputes / genius

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 55,
        "surprise_id": 120,
        "creator_id": 15,
        "genius_id": 22,
        "opened_by": 15,
        "reason": "El trabajo no cumple con lo solicitado",
        "status": "open",
        "created_at": "2026-05-05T12:00:00.000000Z",
        "updated_at": "2026-05-05T12:00:00.000000Z",
        "surprise": { ...datos de la sorpresa... },
        "creator": { ...datos del creador... }
      },
      { ...más disputas donde el usuario es genius... }
    ],
    "last_page": 2,
    "total": 25
  }
}
```

Si el usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

6. CONVERSACIONES Y MENSAJES

6.1 Listar Conversaciones del Usuario (GET /api/conversations)

El endpoint GET /api/conversations devuelve todas las conversaciones en las que el usuario autenticado participa, ya sea como creador o como genius. El backend filtra por creator_id o genius_id, carga las relaciones creator, genius y surprise, y ordena las conversaciones por updated_at en orden descendente. No requiere cuerpo en la petición; solo el token de autenticación.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo token en el header.)

Ejemplo de URL:
/ api / conversations

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
[
{
  "id": 88,
  "surprise_id": 120,
  "creator_id": 15,
  "genius_id": 22,
  "created_at": "2026-05-05T12:00:00.000000Z",
  "updated_at": "2026-05-05T12:30:00.000000Z",
  "creator": { ...datos del creador... },
  "genius": { ...datos del genius... },
  "surprise": { ...datos de la sorpresa... }
},
{ ...más conversaciones del usuario... }
]
```

Si el usuario no está autenticado:
{ "message": "Unauthenticated." }

6.2 Crear u Obtener una Conversación (POST /api/conversations)

El endpoint POST /api/conversations permite crear una conversación entre el creador y el genius de una sorpresa, o devolver la conversación existente si ya fue creada previamente. Solo pueden abrirla el creador o el genius asignado, y la sorpresa debe tener un genius asignado. El backend valida permisos, valida que exista la sorpresa, comprueba si ya existe una conversación y, si no existe, la crea.

Datos que debe enviar el cliente (JSON en texto):
{
 "surprise_id": 120
}

Ejemplo de URL:
/ api / conversations

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta de la API si la conversación NO existía (texto tipo JSON):

```
{
  "id": 88,
  "surprise_id": 120,
  "creator_id": 15,
  "genius_id": 22,
  "created_at": "2026-05-05T12:00:00.000000Z",
  "updated_at": "2026-05-05T12:00:00.000000Z"
}
```

Respuesta de la API si la conversación YA existía:

```
{
  "id": 88,
  "surprise_id": 120,
  "creator_id": 15,
  "genius_id": 22,
  "created_at": "2026-05-01T10:00:00.000000Z",
  "updated_at": "2026-05-05T12:30:00.000000Z"
}
```

Si el usuario NO es creador ni genius de la sorpresa:
{ "error": "Not authorized for this surprise" }

Si la sorpresa NO tiene genius asignado:
{ "error": "No genius assigned to this surprise" }

Si falta surprise_id o no existe:
{ "message": "The surprise_id field is required" }
o
{ "message": "The selected surprise_id is invalid" }

Si la sorpresa no existe:
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }

6.3 Eliminar una Conversación (DELETE /api/conversations/{conversation})

El endpoint DELETE /api/conversations/{conversation} elimina una conversación completa junto con todos sus mensajes y todas las imágenes asociadas. Solo pueden eliminarla el creador o el genius que participan en esa conversación. El backend valida permisos, elimina las imágenes del almacenamiento, borra los mensajes y finalmente borra la conversación.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / conversations / 88

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{  
"message": "Conversation and messages deleted"  
}
```

Si el usuario NO es creador ni genius de la conversación:

```
{ "error": "Not authorized" }
```

Si la conversación no existe:

```
{ "message": "No query results for model [App\\Models\\Conversation] 9999" }
```

6.4 Listar Mensajes de una Conversación (GET /api/conversations/{conversationId}/messages)

El endpoint GET /api/conversations/{conversationId}/messages devuelve todos los mensajes pertenecientes a una conversación específica. Solo pueden verlos el creador o el genius asignados a la sorpresa asociada. El backend valida permisos, carga la conversación, y luego devuelve los mensajes ordenados cronológicamente (ascendente). Cada mensaje incluye la relación sender.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID en la URL y el token en el header.)

Ejemplo de URL:

/ api / conversations / 88 / messages

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
[  
{  
"id": 501,  
"conversation_id": 88,  
"sender_id": 15,  
"content": "Hola, genius",  
"image": null,  
"created_at": "2026-05-05T12:00:00.000000Z",  
"updated_at": "2026-05-05T12:00:00.000000Z",  
"sender": { ...datos del usuario... }  
},  
{  
"id": 502,
```

```
"conversation_id": 88,  
"sender_id": 22,  
"content": "Hola, creador",  
"image": "messages/abc123.jpg",  
"created_at": "2026-05-05T12:01:00.000000Z",  
"updated_at": "2026-05-05T12:01:00.000000Z",  
"sender": { ...datos del usuario... }  
},  
{ ...más mensajes... }  
]
```

Si el usuario NO es creador ni genius de la conversación:
{ "error": "Not authorized" }

Si la conversación no existe:
{ "message": "No query results for model [App\\Models\\Conversation] 9999" }

6.5 Enviar un Mensaje (POST /api/messages)

El endpoint POST /api/messages permite enviar un mensaje dentro de una conversación. Solo pueden enviar mensajes el creador o el genius asignados a la sorpresa. La sorpresa debe estar en estado open, in_progress, delivered o completed. El usuario debe enviar texto o archivo (imagen, vídeo o pdf). El backend valida permisos, estado, longitud máxima, anti-flood (1 mensaje cada 2 segundos), anti-duplicados (no repetir el mismo mensaje en 10 segundos), sanitiza el texto, valida el archivo, guarda el archivo, crea el mensaje, registra auditoría y envía una notificación al otro usuario.

Datos que debe enviar el cliente (JSON en texto):
Ejemplo enviando solo texto:

```
{  
"conversation_id": 88,  
"content": "Hola, genius"  
}
```

Ejemplo enviando archivo (multipart/form-data):
conversation_id: 88
image: archivo.jpg/png/webp/mp4/pdf

Ejemplo enviando texto + archivo (multipart/form-data):
conversation_id: 88
content: "Aquí tienes el archivo"
image: archivo.jpg

Ejemplo de URL:
/ api / messages

Ejemplo de header:
Authorization: Bearer token_de_autenticacion
Content-Type: multipart/form-data (si envía archivo)

Respuesta de la API (texto tipo JSON):
Si envía texto:

```
{
  "id": 501,
  "conversation_id": 88,
  "sender_id": 15,
  "content": "Hola, genius",
  "image": null,
  "created_at": "2026-05-05T12:00:00.000000Z",
  "updated_at": "2026-05-05T12:00:00.000000Z"
}
```

Si envía archivo:

```
{
  "id": 502,
  "conversation_id": 88,
  "sender_id": 15,
  "content": null,
  "image": "messages/archivo_seguro.jpg",
  "created_at": "2026-05-05T12:01:00.000000Z",
  "updated_at": "2026-05-05T12:01:00.000000Z"
}
```

Errores posibles:

Usuario no autorizado:

```
{ "error": "Not authorized" }
```

Estado no permitido:

```
{ "error": "Messages are not allowed in this surprise status" }
```

Sin texto ni archivo:

```
{ "message": "Debes enviar texto o una imagen." }
```

Mensaje demasiado largo:

```
{ "message": "El mensaje es demasiado largo (máx 2000 caracteres)." }
```

Anti-flood:

```
{ "message": "Estás enviando mensajes demasiado rápido." }
```

Anti-duplicados:

```
{ "message": "Ya has enviado este mismo mensaje hace un momento." }
```

Archivo sospechoso:

```
{ "error": "El archivo no coincide con su tipo real. Posible archivo malicioso." }
```

conversation_id inválido:

```
{ "message": "The selected conversation_id is invalid" }
```

Conversación no existe:

```
{ "message": "No query results for model [App\\Models\\Conversation] 9999" }
```

6.6 Enviar un Mensaje desde una Sorpresa (POST /api/surprises/{surpriseId}/messages)

El endpoint POST /api/surprises/{surpriseId}/messages permite enviar un mensaje directamente desde una sorpresa sin conocer la conversación. Solo pueden enviar mensajes el creador o el genius asignado. Si no existe conversación, el sistema la crea automáticamente. La sorpresa debe estar en estado open, in_progress, delivered o completed. El usuario debe enviar texto o archivo (imagen, vídeo o pdf). El backend valida permisos, estado, existencia de genius asignado, crea o reutiliza la conversación y luego reutiliza la lógica del endpoint principal de mensajes.

Datos que debe enviar el cliente (JSON en texto):

Ejemplo enviando solo texto:

```
{
  "content": "Hola, genius"
}
```

Ejemplo enviando archivo (multipart/form-data):

image: archivo.jpg/png/webp/mp4/pdf

Ejemplo enviando texto + archivo (multipart/form-data):

content: "Aquí tienes el archivo"

image: archivo.jpg

Ejemplo de URL:

/ api / surprises / 20 / messages

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Content-Type: multipart/form-data (si envía archivo)

Respuesta de la API (texto tipo JSON):

(Es la misma respuesta que el endpoint POST /api/messages, porque internamente llama a store)

Ejemplo si envía texto:

```
{
  "id": 501,
  "conversation_id": 88,
  "sender_id": 15,
  "content": "Hola, genius",
  "image": null,
  "created_at": "2026-05-05T12:00:00.000000Z",
  "updated_at": "2026-05-05T12:00:00.000000Z"
}
```

Ejemplo si envía archivo:

```
{
  "id": 502,
  "conversation_id": 88,
  "sender_id": 15,
  "content": null,
  "image": "messages/archivo_seguro.jpg",
  "created_at": "2026-05-05T12:01:00.000000Z",
  "updated_at": "2026-05-05T12:01:00.000000Z"
}
```


}

Errores posibles:

Usuario no autorizado para esta sorpresa:

```
{ "error": "Not authorized for this surprise" }
```

La sorpresa no tiene genius asignado:

```
{ "error": "No genius assigned to this surprise" }
```

Estado no permitido:

```
{ "error": "Messages are not allowed in this surprise status" }
```

Sin texto ni archivo:

```
{ "message": "Debes enviar texto o una imagen." }
```

Si la sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

(El resto de errores —mensaje largo, flood, duplicados, archivo malicioso— vienen del endpoint store y se devuelven igual.)

6.7 Descargar archivo de un mensaje (GET /api/messages/file/{id})

El endpoint GET /api/messages/file/{id} permite descargar el archivo adjunto de un mensaje. Solo pueden descargarlo el creador o el genius de la conversación asociada. El backend valida permisos, comprueba que el mensaje tenga archivo, registra auditoría y devuelve la descarga del archivo almacenado en el sistema.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del mensaje en la URL y el token en el header.)

Ejemplo de URL:

/ api / messages / file / 501

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (descarga de archivo):

El servidor devuelve directamente el archivo (jpg, png, webp, mp4, pdf).

No devuelve JSON en caso de éxito, sino la descarga del archivo.

Errores posibles:

Usuario no autorizado:

```
{ "error": "Not authorized" }
```

El mensaje no tiene archivo:

```
{ "error": "No file" }
```

El mensaje no existe:

```
{ "message": "No query results for model [App\\Models\\Message] 9999" }
```

7. OFERTAS Y NEGOCIACIÓN

7.1 Enviar una oferta a una sorpresa (POST /api/surprises/{surpriseId}/offers)

El endpoint POST /api/surprises/{surpriseId}/offers permite que un genius envíe una oferta a una sorpresa. Solo se pueden enviar ofertas cuando la sorpresa está en estado open. El backend valida el precio, el rango permitido según el tamaño de la sorpresa, evita más de una oferta activa por genius, aplica un cooldown de 30 segundos entre ofertas, evita duplicados recientes, sanitiza el mensaje, crea la oferta, registra auditoría y notifica al creador de la sorpresa.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "price": 25.50,
  "message": "Puedo hacerlo en menos de un día",
  "eta_hours": 12
}
```

Ejemplo de URL:

/api/surprises/20/offers

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Offer sent successfully",
  "data": {
    "id": 301,
    "surprise_id": 20,
    "genius_id": 15,
    "price": 25.50,
    "original_price": 25.50,
    "message": "Puedo hacerlo en menos de un día",
    "eta_hours": 12,
    "status": "pending",
    "creator_bid_count": 0,
    "genius_bid_count": 0,
    "created_at": "2026-05-05T12:30:00.000000Z",
    "updated_at": "2026-05-05T12:30:00.000000Z"
  }
}
```

Errores posibles:

La sorpresa no está open:

```
{ "error": "You can only send offers for open surprises" }
```

Precio fuera del rango permitido:

```
{ "error": "Price is not realistic for this surprise type" }
```

El genius ya tiene una oferta activa:

```
{ "error": "You already have an active offer for this surprise" }
```

Cooldown de 30 segundos:

```
{ "error": "You are sending offers too fast. Please wait a bit." }
```

Oferta duplicada reciente:

```
{ "error": "You already sent an identical offer recently" }
```

Usuario no autorizado para esta sorpresa:

```
{ "error": "Not authorized for this surprise" }
```

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

7.2 Listar ofertas de una sorpresa (GET /api/surprises/{surpriseId}/offers)

El endpoint GET /api/surprises/{surpriseId}/offers devuelve todas las ofertas enviadas a una sorpresa concreta. No requiere autenticación especial más allá del token, ya que solo lista las ofertas. El backend obtiene todas las ofertas asociadas a la sorpresa, incluyendo la información del genius que las envió y el historial de bids (negociaciones), y devuelve el listado completo.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la sorpresa en la URL y el token en el header.)

Ejemplo de URL:

/ api / surprises / 20 / offers

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 301,
      "surprise_id": 20,
      "genius_id": 15,
      "price": 25.50,
      "original_price": 25.50,
      "message": "Puedo hacerlo en menos de un día",
      "eta_hours": 12,
      "status": "pending",
      "creator_bid_count": 0,
      "genius_bid_count": 0,
      "created_at": "2026-05-05T12:30:00.000000Z",
      "updated_at": "2026-05-05T12:30:00.000000Z",
      "genius": { ...datos del usuario... },
      "bids": [ ...historial de negociación... ]
    }
  ]
}
```

```

},
{
  "id": 302,
  "surprise_id": 20,
  "genius_id": 18,
  "price": 30.00,
  "original_price": 30.00,
  "message": null,
  "eta_hours": 24,
  "status": "pending",
  "creator_bid_count": 0,
  "genius_bid_count": 0,
  "created_at": "2026-05-05T12:45:00.000000Z",
  "updated_at": "2026-05-05T12:45:00.000000Z",
  "genius": { ...datos del usuario... },
  "bids": []
}
]
}

```

Errores posibles:

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

7.3 Enviar una contraoferta en una negociación (POST /api/offers/{offerId}/counter)

El endpoint POST /api/offers/{offerId}/counter permite que el creador o el genius envíen una contraoferta dentro de una negociación activa. Cada usuario tiene un máximo de 5 contraofertas y la negociación completa un máximo de 10 rondas. El backend valida permisos, rol del usuario, límites de bids, cooldown de 5 segundos, rango de precio permitido según el tamaño de la sorpresa, sanitiza el mensaje, crea un nuevo bid, actualiza la oferta con los nuevos valores, registra auditoría y notifica a la otra parte.

Datos que debe enviar el cliente (JSON en texto):

```

{
  "price": 22.00,
  "eta_hours": 10,
  "message": "Podría ajustarlo un poco más"
}

```

Ejemplo de URL:

/ api / offers / 301 / counter

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```

{
  "success": true,
  "message": "Counteroffer sent successfully",
  "data": {

```

```

"offer": {
  "id": 301,
  "surprise_id": 20,
  "genius_id": 15,
  "price": 22.00,
  "eta_hours": 10,
  "message": "Podría ajustarlo un poco más",
  "status": "negotiating",
  "creator_bid_count": 1,
  "genius_bid_count": 0,
  "created_at": "2026-05-05T12:30:00.000000Z",
  "updated_at": "2026-05-05T12:35:00.000000Z"
},
"bid": {
  "id": 9001,
  "offer_id": 301,
  "user_id": 22,
  "role": "creator",
  "price": 22.00,
  "eta_hours": 10,
  "message": "Podría ajustarlo un poco más",
  "created_at": "2026-05-05T12:35:00.000000Z",
  "updated_at": "2026-05-05T12:35:00.000000Z"
}
}
}
}

```

Errores posibles:

Usuario no autorizado para esta negociación:

```
{ "error": "Not authorized for this negotiation" }
```

El creador ya alcanzó el máximo de contraofertas:

```
{ "error": "Creator has reached the maximum number of counteroffers for this offer" }
```

El genius ya alcanzó el máximo de contraofertas:

```
{ "error": "Genius has reached the maximum number of counteroffers for this offer" }
```

La negociación ya alcanzó 10 rondas:

```
{ "error": "This negotiation has reached the maximum number of rounds" }
```

Cooldown de 5 segundos entre contraofertas:

```
{ "error": "You are sending counteroffers too fast" }
```

Precio fuera del rango permitido:

```
{ "error": "Price is not realistic for this surprise type" }
```

Oferta no existe:

```
{ "message": "No query results for model [App\\Models\\Offer] 9999" }
```

7.4 Aceptar una oferta (POST /api/offers/{offerId}/accept)

El endpoint POST /api/offers/{offerId}/accept permite que el creador de la sorpresa acepte una oferta enviada por un genius. Solo el creador puede aceptar una oferta. Al aceptar, la oferta pasa a estado accepted, la sorpresa asigna al genius, cambia su estado a in_progress, guarda el precio inicial del genius y el precio final tras la negociación, rechaza automáticamente todas las demás ofertas, envía notificaciones al genius ganador y a los demás participantes, y registra auditoría.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la oferta en la URL y el token en el header.)

Ejemplo de URL:

/api / offers / 301 / accept

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Offer accepted and genius assigned",
  "data": {
    "id": 301,
    "surprise_id": 20,
    "genius_id": 15,
    "price": 22.00,
    "original_price": 25.50,
    "message": "Podría ajustarlo un poco más",
    "eta_hours": 10,
    "status": "accepted",
    "creator_bid_count": 1,
    "genius_bid_count": 0,
    "created_at": "2026-05-05T12:30:00.000000Z",
    "updated_at": "2026-05-05T12:40:00.000000Z"
  }
}
```

Errores posibles:

Usuario no autorizado (solo el creador puede aceptar):

```
{ "error": "Not authorized" }
```

Oferta no existe:

```
{ "message": "No query results for model [App\\Models\\Offer] 9999" }
```

8. ARCHIVOS

8.1 Subir un archivo a una sorpresa (POST /api/surprises/{id}/files)

El endpoint POST /api/surprises/{id}/files permite que el creador o el genius asignado suban archivos relacionados con la sorpresa. Solo se pueden subir archivos cuando la sorpresa está en estado in_progress o delivered. El backend valida permisos, estado, tipo de archivo, tamaño máximo, MIME real, guarda el archivo, registra auditoría y envía notificaciones dependiendo de quién lo suba.

Datos que debe enviar el cliente (JSON en texto o multipart/form-data):

Ejemplo multipart enviando archivo:

file: archivo.jpg/png/webp/pdf/mp4/mp3

Ejemplo de campos:

file: archivo.pdf

(no requiere más campos, el ID va en la URL)

Ejemplo de URL:

/ api / surprises / 20 / files

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Content-Type: multipart/form-data

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "File uploaded successfully",
  "data": {
    "id": 900,
    "surprise_id": 20,
    "filename": "documento.pdf",
    "path": "surprises/20/documento.pdf",
    "mime": "application/pdf",
    "size": 204800,
    "file_url": "https://tuapp.com/storage/surprises/20/documento.pdf",
    "file_type": "application",
    "created_at": "2026-05-05T13:00:00.000000Z",
    "updated_at": "2026-05-05T13:00:00.000000Z"
  }
}
```

Errores posibles:

Usuario no autorizado:

```
{ "error": "No autorizado" }
```

Estado no permitido para subir archivos:

```
{ "error": "No se pueden subir archivos en este estado" }
```

Archivo no enviado o inválido:

```
{ "message": "The file field is required" }
```

Tipo de archivo no permitido:

```
{ "error": "Tipo de archivo no permitido" }
```

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

8.2 Listar archivos de una sorpresa (GET /api/surprises/{id}/files)

El endpoint GET /api/surprises/{id}/files devuelve todos los archivos que han sido subidos a una sorpresa. No requiere enviar datos en el cuerpo, solo el ID de la sorpresa en la URL. El backend obtiene todos los registros de archivos asociados a la sorpresa y devuelve la lista completa con su información: nombre, ruta, tipo MIME, tamaño y URL pública.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la sorpresa en la URL y el token en el header.)

Ejemplo de URL:

/ api / surprises / 20 / files

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 900,
      "surprise_id": 20,
      "filename": "documento.pdf",
      "path": "surprises/20/documento.pdf",
      "mime": "application/pdf",
      "size": 204800,
      "file_url": "https://tuapp.com/storage/surprises/20/documento.pdf",
      "file_type": "application",
      "created_at": "2026-05-05T13:00:00.000000Z",
      "updated_at": "2026-05-05T13:00:00.000000Z"
    },
    {
      "id": 901,
      "surprise_id": 20,
      "filename": "imagen.png",
      "path": "surprises/20/imagen.png",
      "mime": "image/png",
      "size": 102400,
      "file_url": "https://tuapp.com/storage/surprises/20/imagen.png",
      "file_type": "image",
      "created_at": "2026-05-05T13:10:00.000000Z",
      "updated_at": "2026-05-05T13:10:00.000000Z"
    }
  ]
}
```



```
]
}
```

Errores posibles:

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

8.3 Descargar un archivo de sorpresa (GET /api/files/{id}/download)

El endpoint GET /api/files/{id}/download permite descargar un archivo previamente subido a una sorpresa. No requiere enviar datos en el cuerpo, solo el ID del archivo en la URL. El backend busca el archivo, comprueba que exista físicamente en el almacenamiento y devuelve la descarga con el nombre y tipo MIME correctos. Si el archivo no existe en disco, devuelve un error.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del archivo en la URL y el token en el header.)

Ejemplo de URL:

/ api / files / 900 / download

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (descarga de archivo):

El servidor devuelve directamente el archivo (jpg, png, webp, pdf, mp4, mp3).

No devuelve JSON en caso de éxito, sino la descarga del archivo.

Errores posibles:

Archivo no encontrado en disco:

```
{ "error": "File not found" }
```

Archivo no existe en la base de datos:

```
{ "message": "No query results for model [App\\Models\\SurpriseFile] 9999" }
```

9. NOTIFICACIONES

9.1 Listar notificaciones de un usuario (GET /api/users/{id}/notifications)

El endpoint GET /api/users/{id}/notifications devuelve todas las notificaciones pertenecientes a un usuario. No requiere enviar datos en el cuerpo, solo el ID del usuario en la URL. El backend obtiene todas las notificaciones ordenadas por fecha de creación en orden descendente y devuelve la lista completa.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/api / users / 15 / notifications

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 7001,
      "user_id": 15,
      "title": "Nueva oferta recibida",
      "message": "Un genius ha enviado una oferta a tu sorpresa",
      "metadata": null,
      "type": "info",
      "read_flag": 0,
      "created_at": "2026-05-05T14:00:00.000000Z",
      "updated_at": "2026-05-05T14:00:00.000000Z"
    },
    {
      "id": 7000,
      "user_id": 15,
      "title": "Archivo subido",
      "message": "El genius ha subido un archivo a tu sorpresa",
      "metadata": null,
      "type": "info",
      "read_flag": 1,
      "created_at": "2026-05-05T13:50:00.000000Z",
      "updated_at": "2026-05-05T13:50:00.000000Z"
    }
  ]
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

9.2 Marcar una notificación como leída (POST /api/notifications/{id}/read)

El endpoint POST /api/notifications/{id}/read permite marcar una notificación como leída. No requiere enviar datos en el cuerpo, solo el ID de la notificación en la URL. El backend busca la notificación, actualiza el campo read_flag a 1 y devuelve una respuesta de éxito.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la notificación en la URL y el token en el header.)

Ejemplo de URL:

/ api / notifications / 7001 / read

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
"success": true,
"message": "Notification marked as read"
}
```

Errores posibles:

Notificación no existe:

```
{ "message": "No query results for model [App\\Models\\Notification] 9999" }
```

9.3 Marcar todas las notificaciones como leídas (POST /api/users/{id}/notifications/read-all)

El endpoint POST /api/users/{id}/notifications/read-all permite marcar todas las notificaciones de un usuario como leídas. No requiere enviar datos en el cuerpo, solo el ID del usuario en la URL. El backend actualiza todas las notificaciones del usuario estableciendo read_flag en 1 y devuelve una respuesta de éxito.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 15 / notifications / read-all

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
"success": true,
"message": "All notifications marked as read"
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

10. REVIEWS

10.1 Crear una review de una sorpresa (POST /api/surprises/{id}/review)

El endpoint POST /api/surprises/{id}/review permite que el creador de la sorpresa deje una valoración cuando la sorpresa ha sido completada. Solo el creador puede valorar y solo puede hacerse una review por sorpresa. El backend valida el estado, permisos, evita duplicados, valida los datos, sanitiza el comentario, crea la review, recalcula el rating del genius, actualiza su nivel, envía notificación y devuelve la review creada.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "rating_surprise": 5,
  "rating_genius": 4,
  "comment": "Muy buen trabajo, comunicación excelente"
}
```

Ejemplo de URL:

/ api / surprises / 20 / review

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Review created successfully",
  "data": {
    "id": 5001,
    "surprise_id": 20,
    "reviewer_id": 15,
    "reviewed_user_id": 33,
    "rating_surprise": 5,
    "rating_genius": 4,
    "comment": "Muy buen trabajo, comunicación excelente",
    "created_at": "2026-05-05T14:10:00.000000Z",
    "updated_at": "2026-05-05T14:10:00.000000Z"
  }
}
```

Errores posibles:

La sorpresa no está completada:

```
{ "error": "Surprise must be completed before reviewing" }
```

Solo el creador puede valorar:

```
{ "error": "Only the creator can review this surprise" }
```

La sorpresa ya tiene una review:

```
{ "error": "This surprise already has a review" }
```

Datos inválidos:

```
{ "message": "The rating_surprise field is required" }
```

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

10.2 Listar reviews recibidas por un usuario (GET /api/users/{id}/reviews)

El endpoint GET /api/users/{id}/reviews devuelve todas las reviews que ha recibido un usuario como genius. No requiere enviar datos en el cuerpo, solo el ID del usuario en la URL. El backend obtiene todas las reviews donde el usuario fue el reviewed_user_id, incluyendo información del reviewer y de la sorpresa asociada, ordenadas por fecha descendente.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / reviews

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 5001,
      "surprise_id": 20,
      "reviewer_id": 15,
      "reviewed_user_id": 33,
      "rating_surprise": 5,
      "rating_genius": 4,
      "comment": "Muy buen trabajo, comunicación excelente",
      "created_at": "2026-05-05T14:10:00.000000Z",
      "updated_at": "2026-05-05T14:10:00.000000Z",
      "reviewer": { ...datos del usuario... },
      "surprise": { ...datos de la sorpresa... }
    },
    {
      "id": 5000,
      "surprise_id": 18,
      "reviewer_id": 12,
      "reviewed_user_id": 33,
      "rating_surprise": 4,
      "rating_genius": 5,
      "comment": null,
      "created_at": "2026-05-04T18:00:00.000000Z",
      "updated_at": "2026-05-04T18:00:00.000000Z",
      "reviewer": { ...datos del usuario... },
      "surprise": { ...datos de la sorpresa... }
    }
  ]
}
```

```
}  
]  
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

10.3 Obtener rating promedio de un usuario (GET /api/users/{id}/rating)

El endpoint GET /api/users/{id}/rating devuelve la media de las valoraciones recibidas por un usuario como genius. No requiere enviar datos en el cuerpo, solo el ID del usuario en la URL. El backend calcula el promedio de rating_genius en todas las reviews donde el usuario fue el reviewed_user_id. Si no tiene reviews, devuelve 0.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / rating

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{  
  "success": true,  
  "rating": 4.25  
}
```

(Si no tiene reviews, rating = 0)

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11. SKILLS

11.1 Listar todas las skills del sistema (GET /api/skills)

El endpoint GET /api/skills devuelve el listado completo de skills disponibles en la plataforma. No requiere enviar datos en el cuerpo ni parámetros adicionales. El backend simplemente obtiene todas las skills registradas en la base de datos y las devuelve en un array.

Datos que debe enviar el cliente (JSON en texto):
(No requiere cuerpo. Solo se llama al endpoint con el token en el header.)

Ejemplo de URL:
/ api / skills

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Fotografía",
      "category": "Creatividad",
      "description": "Captura y edición de imágenes",
      "created_at": "2026-04-20T10:00:00.000000Z",
      "updated_at": "2026-04-20T10:00:00.000000Z"
    },
    {
      "id": 2,
      "name": "Decoración",
      "category": "Hogar",
      "description": "Diseño y ambientación de espacios",
      "created_at": "2026-04-20T10:00:00.000000Z",
      "updated_at": "2026-04-20T10:00:00.000000Z"
    }
  ]
}
```

Errores posibles:
(No aplica ningún error específico más allá de errores internos del servidor.)

11.2 Listar skills propuestas y activas de un usuario (GET /api/users/{id}/skills)

El endpoint GET /api/users/{id}/skills devuelve todas las skills asociadas a un usuario, separadas en dos grupos: skills propuestas (nivel 0, aún no activas) y skills activas (con

nivel y XP real). El backend obtiene al usuario junto con sus relaciones proposedSkills y activeSkills, transforma cada grupo al formato esperado y devuelve ambos listados.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / skills

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "proposed": [
      {
        "id": 10,
        "name": "Fotografía",
        "category": "Creatividad",
        "level": 0,
        "status": "proposed"
      },
      {
        "id": 14,
        "name": "Decoración",
        "category": "Hogar",
        "level": 0,
        "status": "proposed"
      }
    ],
    "active": [
      {
        "id": 3,
        "name": "Edición de video",
        "category": "Creatividad",
        "level": 2,
        "xp": 140,
        "status": "active"
      },
      {
        "id": 7,
        "name": "Organización",
        "category": "Habilidades prácticas",
        "level": 1,
        "xp": 20,
        "status": "active"
      }
    ]
  }
}
```


Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11.3 Actualizar las skills propuestas de un usuario (POST /api/users/{id}/proposed-skills)

El endpoint POST /api/users/{id}/proposed-skills permite actualizar la lista de skills propuestas por un usuario. Las skills propuestas representan habilidades que el usuario declara tener pero que aún no están activas. El backend valida que el array de skills exista y que cada ID corresponda a una skill válida, obtiene al usuario y sincroniza la relación proposedSkills reemplazando las anteriores.

Datos que debe enviar el cliente (JSON en texto):

```
{  
  "skills": [10, 14, 22]  
}
```

Ejemplo de URL:

/api / users / 33 / proposed-skills

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{  
  "success": true,  
  "message": "Proposed skills updated"  
}
```

Errores posibles:

El array no se envió o está vacío:

```
{ "message": "The skills field is required" }
```

Una skill no existe:

```
{ "message": "The selected skills.0 is invalid" }
```

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11.4 Obtener el top 10 de genios por skill (GET /api/skills/{skillId}/top)

El endpoint GET /api/skills/{skillId}/top devuelve el ranking de los 10 mejores genios que tienen activada una skill concreta. El backend valida que la skill exista, obtiene los usuarios con esa skill activa, calcula su nivel, XP y número de sorpresas completadas con esa skill, los ordena por nivel y XP, y devuelve el top 10.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la skill en la URL y el token en el header.)

Ejemplo de URL:
/ api / skills / 3 / top

Ejemplo de header:
Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "skill": "Fotografía",
  "top10": [
    {
      "user_id": 33,
      "name": "Laura Gómez",
      "level": 4,
      "xp": 820,
      "completed_surprises": 12
    },
    {
      "user_id": 15,
      "name": "Carlos Ruiz",
      "level": 3,
      "xp": 450,
      "completed_surprises": 5
    },
    {
      "user_id": 48,
      "name": "Ana Torres",
      "level": 3,
      "xp": 300,
      "completed_surprises": 3
    }
  ]
}
```

Errores posibles:

Skill no existe:

```
{ "message": "No query results for model [App\\Models\\Skill] 9999" }
```

11.5 Ranking de genios por número de sorpresas completadas en una skill (GET /api/skills/{skillId}/ranking)

El endpoint GET /api/skills/{skillId}/ranking devuelve un ranking basado en la cantidad de sorpresas completadas por cada genius dentro de una skill específica. El backend valida que la skill exista, cuenta cuántas sorpresas completó cada genius con esa skill, calcula un nivel basado en ese total, ordena los resultados y devuelve la lista completa con posición, usuario, total completado y nivel.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la skill en la URL y el token en el header.)

Ejemplo de URL:

/ api / skills / 3 / ranking

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "skill": "Fotografía",
  "ranking": [
    {
      "position": 1,
      "user_id": 33,
      "user_name": "Laura Gómez",
      "completed": 52,
      "level": 5
    },
    {
      "position": 2,
      "user_id": 15,
      "user_name": "Carlos Ruiz",
      "completed": 27,
      "level": 4
    },
    {
      "position": 3,
      "user_id": 48,
      "user_name": "Ana Torres",
      "completed": 11,
      "level": 3
    }
  ]
}
```

Errores posibles:

Skill no existe:

```
{ "message": "No query results for model [App\\Models\\Skill] 9999" }
```

11.6 Progreso de un usuario en una skill (GET /api/users/{userId}/skills/{skillId}/progress)

El endpoint GET /api/users/{userId}/skills/{skillId}/progress devuelve el progreso del usuario como genius dentro de una skill concreta. El backend valida que el usuario y la skill existan, cuenta cuántas sorpresas completó el usuario con esa skill, determina el nivel actual según el total, calcula cuántas faltan para el siguiente nivel, el porcentaje de progreso y devuelve toda la información estructurada.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasan los IDs en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / skills / 3 / progress

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "skill_id": 3,
    "skill_name": "Fotografía",
    "level": 2,
    "completed": 6,
    "next_level_at": 10,
    "remaining": 4,
    "progress_percentage": 60.0
  }
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

Skill no existe:

```
{ "message": "No query results for model [App\\Models\\Skill] 9999" }
```

11.7 Progreso del usuario en todas sus skills activas (GET /api/users/{userId}/skills/progress)

El endpoint GET /api/users/{userId}/skills/progress devuelve el progreso del usuario como genius en todas las skills que tiene activas. El backend valida que el usuario exista, obtiene todas sus skills activas, calcula cuántas sorpresas completó en cada skill, determina el nivel actual, cuántas faltan para el siguiente nivel, el porcentaje de progreso y devuelve un listado con una entrada por skill.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasan los IDs en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / skills / progress

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "skill_id": 3,
      "skill_name": "Fotografía",
      "level": 2,
      "completed": 6,
```

```

"next_level_at": 10,
"remaining": 4,
"progress_percentage": 60.0
},
{
"skill_id": 7,
"skill_name": "Organización",
"level": 1,
"completed": 2,
"next_level_at": 4,
"remaining": 2,
"progress_percentage": 50.0
}
]
}

```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11.8 Historial de sorpresas completadas por skill (GET /api/users/{userId}/skills/{skillId}/history)

El endpoint GET /api/users/{userId}/skills/{skillId}/history devuelve el historial completo de todas las sorpresas que un usuario ha realizado como genius dentro de una skill concreta. El backend obtiene todas las sorpresas filtradas por genius_id y skill_id, incluye información del creador, archivos y reviews asociadas, las ordena por fecha descendente y devuelve la lista completa.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasan los IDs en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / skills / 3 / history

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```

{
"success": true,
"data": [
{
"id": 120,
"creator_id": 15,
"genius_id": 33,
"skill_id": 3,
"title": "Sesión de fotos en exterior",
"status": "completed",
"created_at": "2026-05-01T10:00:00.000000Z",
"updated_at": "2026-05-01T12:00:00.000000Z",
"creator": { ...datos del usuario... },

```

```

"files": [ ...archivos subidos... ],
"reviews": [ ...reviews asociadas... ]
},
{
"id": 118,
"creator_id": 22,
"genius_id": 33,
"skill_id": 3,
"title": "Retrato profesional",
"status": "completed",
"created_at": "2026-04-20T16:00:00.000000Z",
"updated_at": "2026-04-20T17:00:00.000000Z",
"creator": { ...datos del usuario... },
"files": [ ...archivos subidos... ],
"reviews": [ ...reviews asociadas... ]
}
]
}

```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

Skill no existe:

```
{ "message": "No query results for model [App\\Models\\Skill] 9999" }
```

11.9 Nivel global de un usuario como genius (GET /api/users/{userId}/level)

El endpoint GET /api/users/{userId}/level calcula el nivel global del usuario como genius teniendo en cuenta todas sus skills activas. El backend valida que el usuario exista, obtiene todas sus skills, calcula cuántas sorpresas completó en cada una, determina el nivel individual por skill, aplica un promedio ponderado según el número de sorpresas completadas y devuelve el nivel global. Si el usuario no tiene skills activas, el nivel global es 1.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / level

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```

{
"success": true,
"global_level": 2.87
}

```

(Si el usuario no tiene skills activas, global_level = 1)

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11.10 Dashboard de skills del usuario (GET /api/users/{userId}/skills/dashboard)

El endpoint GET /api/users/{userId}/skills/dashboard devuelve un resumen global del rendimiento del usuario como genius. Combina dos fuentes internas: el progreso detallado en todas las skills activas (allProgress) y el nivel global calculado a partir de todas ellas (globalLevel). El backend obtiene ambos valores llamando internamente a esos métodos y devuelve un único objeto con toda la información consolidada.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token en el header.)

Ejemplo de URL:

/ api / users / 33 / skills / dashboard

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "progress": [
    {
      "skill_id": 3,
      "skill_name": "Fotografía",
      "level": 2,
      "completed": 6,
      "next_level_at": 10,
      "remaining": 4,
      "progress_percentage": 60.0
    },
    {
      "skill_id": 7,
      "skill_name": "Organización",
      "level": 1,
      "completed": 2,
      "next_level_at": 4,
      "remaining": 2,
      "progress_percentage": 50.0
    }
  ],
  "global_level": 2.87
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

11.11 Sugerencias de genios para una skill (GET /api/skills/{skillId}/genius-suggestions)

El endpoint GET /api/skills/{skillId}/genius-suggestions devuelve una lista ordenada de genios que tienen activada una skill concreta. El backend obtiene todos los usuarios que poseen esa skill, calcula su reputación media, el número de sorpresas completadas, su nivel y progreso dentro de la skill, y luego ordena los resultados por nivel, rating y número de sorpresas completadas.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la skill en la URL y el token en el header.)

Ejemplo de URL:

/ api / skills / 3 / genius-suggestions

Ejemplo de header:

Authorization: Bearer token_de_autenticacion

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": [
    {
      "user_id": 33,
      "name": "Laura Gómez",
      "avatar": "https://cdn.surpriser.app/avatars/laura.png",
      "skill_level": 4,
      "skill_progress": 820,
      "rating": 4.8,
      "completed_surprises": 12
    },
    {
      "user_id": 15,
      "name": "Carlos Ruiz",
      "avatar": "https://cdn.surpriser.app/avatars/carlos.png",
      "skill_level": 3,
      "skill_progress": 450,
      "rating": 4.5,
      "completed_surprises": 5
    }
  ]
}
```

Errores posibles:

Skill no existe:

```
{ "message": "No query results for model [App\\Models\\Skill] 9999" }
```


12. ADMIN

12.1 Dashboard general del panel de administración (GET /api/admin/dashboard)

El endpoint GET /api/admin/dashboard devuelve las métricas globales del sistema para el panel de administración. El backend calcula el total de usuarios, total de genios, total de creadores, total de sorpresas creadas, número de disputas abiertas, sorpresas completadas y sorpresas canceladas, y devuelve toda la información en un único objeto.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se llama al endpoint con el token de administrador en el header.)

Ejemplo de URL:

/ api / admin / dashboard

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "total_users": 1520,
    "total_genius": 430,
    "total_creator": 1090,
    "total_surprises": 2840,
    "pending_disputes": 7,
    "completed_surprises": 1980,
    "cancelled_surprises": 210
  }
}
```

Errores posibles:

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.2 Listar disputas con filtros avanzados (GET /api/admin/disputes)

El endpoint GET /api/admin/disputes permite a un administrador obtener el listado completo de disputas del sistema, con múltiples filtros opcionales. El backend carga cada disputa junto con la sorpresa asociada, el creador y el genio, aplica los filtros enviados por query params, ordena por fecha descendente y devuelve los resultados paginados (20 por página).

Filtros disponibles (query params):

- status = open | resolved
- winner = creator | genius
- creator_id = ID del creador

- genius_id = ID del genio
- surprise_id = ID de la sorpresa
- date_from = fecha mínima (YYYY-MM-DD)
- date_to = fecha máxima (YYYY-MM-DD)
- search = texto libre en reason o resolution

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se envían filtros opcionales por query params.)

Ejemplo de URL:

/api/admin/disputes?status=open&creator_id=15&search=entrega

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 120,
        "surprise_id": 55,
        "creator_id": 15,
        "genius_id": 33,
        "opened_by": 15,
        "reason": "El genio entregó tarde y no cumplió requisitos",
        "status": "open",
        "winner": null,
        "resolution": null,
        "created_at": "2026-05-05T10:00:00.000000Z",
        "updated_at": "2026-05-05T10:00:00.000000Z",
        "surprise": { ...datos... },
        "creator": { ...datos... },
        "genius": { ...datos... }
      }
    ],
    "last_page": 3,
    "total": 45
  }
}
```

Errores posibles:

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.3 Resolver una disputa (POST /api/admin/disputes/{id}/resolve)

El endpoint POST /api/admin/disputes/{id}/resolve permite a un administrador resolver una disputa abierta. El backend valida que el usuario autenticado sea administrador, verifica

que la disputa esté en estado open, valida los campos enviados, actualiza el estado a resolved, registra la resolución, aplica penalización automática si corresponde, genera un registro de auditoría y envía notificaciones a las partes involucradas.

Datos que debe enviar el cliente (JSON en texto):

```
{
  "winner": "creator",
  "resolution": "El genio no entregó el trabajo solicitado"
}
```

Valores permitidos para winner:

- creator
- genius
- none

Ejemplo de URL:

/ api / admin / disputes / 120 / resolve

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "message": "Dispute resolved successfully",
  "data": {
    "id": 120,
    "surprise_id": 55,
    "creator_id": 15,
    "genius_id": 33,
    "opened_by": 15,
    "reason": "El genio entregó tarde y no cumplió requisitos",
    "status": "resolved",
    "winner": "creator",
    "resolution": "El genio no entregó el trabajo solicitado",
    "resolved_at": "2026-05-05T14:20:00.000000Z",
    "created_at": "2026-05-05T10:00:00.000000Z",
    "updated_at": "2026-05-05T14:20:00.000000Z",
    "surprise": { ...datos... },
    "creator": { ...datos... },
    "genius": { ...datos... }
  }
}
```

Errores posibles:

No es administrador:

```
{ "error": "Not authorized" }
```

La disputa ya está resuelta:

```
{ "error": "Dispute already resolved" }
```

Campos inválidos:

```
{ "message": "The winner field is required" }
```

Disputa no existe:

```
{ "message": "No query results for model [App\\Models\\Dispute] 9999" }
```

12.4 Listar usuarios del sistema (GET /api/admin/users)

El endpoint GET /api/admin/users devuelve el listado completo de usuarios registrados en la plataforma, ordenados por fecha de creación descendente. El backend aplica paginación automática (20 usuarios por página) y devuelve toda la información necesaria para construir un listado en el panel de administración.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se llama al endpoint con el token de administrador en el header.)

Ejemplo de URL:

/ api / admin / users ? page=1

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{
  "success": true,
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 120,
        "name": "Laura Gómez",
        "email": "laura@example.com",
        "avatar": "https://cdn.surpriser.app/avatars/laura.png",
        "is_creator": 1,
        "is_genius": 0,
        "created_at": "2026-05-05T10:00:00.000000Z",
        "updated_at": "2026-05-05T10:00:00.000000Z"
      },
      {
        "id": 119,
        "name": "Carlos Ruiz",
        "email": "carlos@example.com",
        "avatar": "https://cdn.surpriser.app/avatars/carlos.png",
        "is_creator": 0,
        "is_genius": 1,
        "created_at": "2026-05-04T18:00:00.000000Z",
        "updated_at": "2026-05-04T18:00:00.000000Z"
      }
    ]
  }
}
```

```
"last_page": 12,  
"total": 240  
}  
}
```

Errores posibles:

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.5 Banear a un usuario (POST /api/admin/users/{id}/ban)

El endpoint POST /api/admin/users/{id}/ban permite a un administrador marcar a un usuario como baneado dentro de la plataforma. El backend valida que el usuario exista, actualiza el campo banned a 1 y guarda el cambio. No requiere cuerpo en la petición.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token de administrador en el header.)

Ejemplo de URL:

```
/ api / admin / users / 120 / ban
```

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{  
"success": true,  
"message": "User banned successfully"  
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.6 Desbanear a un usuario (POST /api/admin/users/{id}/unban)

El endpoint POST /api/admin/users/{id}/unban permite a un administrador restaurar el acceso de un usuario previamente baneado. El backend valida que el usuario exista, actualiza el campo banned a 0 y guarda el cambio. No requiere cuerpo en la petición.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID del usuario en la URL y el token de administrador en el header.)

Ejemplo de URL:

```
/ api / admin / users / 120 / unban
```

Ejemplo de header:

Authorization: Bearer token_admin
Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{  
  "success": true,  
  "message": "User unbanned successfully"  
}
```

Errores posibles:

Usuario no existe:

```
{ "message": "No query results for model [App\\Models\\User] 9999" }
```

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.7 Listar todas las sorpresas del sistema (GET /api/admin/surprises)

El endpoint GET /api/admin/surprises devuelve el listado completo de sorpresas creadas en la plataforma, ordenadas por fecha de creación descendente. El backend incluye la información del creador y del genio asignado (si existe) y aplica paginación automática de 20 elementos por página.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se llama al endpoint con el token de administrador en el header.)

Ejemplo de URL:

/ api / admin / surprises ? page=1

Ejemplo de header:

Authorization: Bearer token_admin
Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```
{  
  "success": true,  
  "data": {  
    "current_page": 1,  
    "data": [  
      {  
        "id": 220,  
        "title": "Sesión de fotos sorpresa",  
        "creator_id": 15,  
        "genius_id": 33,  
        "skill_id": 3,  
        "status": "completed",  
        "created_at": "2026-05-05T10:00:00.000000Z",  
        "updated_at": "2026-05-05T12:00:00.000000Z",  
        "creator": { ...datos... },  
        "genius": { ...datos... }  
      }  
    ]  
  }  
}
```

```

},
{
  "id": 219,
  "title": "Carta personalizada",
  "creator_id": 22,
  "genius_id": null,
  "skill_id": 7,
  "status": "pending",
  "created_at": "2026-05-04T18:00:00.000000Z",
  "updated_at": "2026-05-04T18:00:00.000000Z",
  "creator": { ...datos... },
  "genius": null
}
],
"last_page": 14,
"total": 280
}
}

```

Errores posibles:

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```

12.8 Cancelación forzada de una sorpresa por un administrador (POST /api/admin/surprises/{id}/force-cancel)

El endpoint POST /api/admin/surprises/{id}/force-cancel permite a un administrador cancelar manualmente cualquier sorpresa del sistema, independientemente de su estado actual. El backend valida que la sorpresa exista, comprueba que no esté ya cancelada, actualiza el estado a cancelled, marca que fue cancelada por un administrador y guarda los cambios.

Datos que debe enviar el cliente (JSON en texto):

(No requiere cuerpo. Solo se pasa el ID de la sorpresa en la URL y el token de administrador en el header.)

Ejemplo de URL:

/ api / admin / surprises / 220 / force-cancel

Ejemplo de header:

Authorization: Bearer token_admin

Content-Type: application/json

Respuesta de la API (texto tipo JSON):

```

{
  "success": true,
  "message": "Surprise cancelled by admin"
}

```

Errores posibles:

Sorpresa ya cancelada:

```
{ "error": "Already cancelled" }
```

Sorpresa no existe:

```
{ "message": "No query results for model [App\\Models\\Surprise] 9999" }
```

Token inválido o sin permisos de admin:

```
{ "message": "Unauthorized" }
```


RESUMEN GENERAL

1. REGISTRO, LOGIN, LOGOUT Y RECUPERACIÓN

- 1.1 Registro de Usuario — POST /api/register — JSON {name, email, password, password_confirmation}
- 1.2 Inicio de Sesión — POST /api/login — JSON {email, password}
- 1.3 Solicitud de Recuperación — POST /api/forgot-password — JSON {email}
- 1.4 Restablecer Contraseña — POST /api/reset-password — JSON {token, email, password, password_confirmation}
- 1.5 Cerrar Sesión — POST /api/logout — nada (Header: Bearer token)

2. LANDING PAGE

- 2.1 Suscripción Landing — POST /api/landing/subscribe — JSON {email}

3. USUARIOS

- 3.1 Obtener Usuario Autenticado — GET /api/me — nada (Header: Bearer token)
- 3.2 Actualizar Perfil — PUT /api/user/profile — JSON opcional {name, username, bio, phone, location_city, location_country, avatar} (Header: Bearer token)
- 3.3 Dashboard de Usuario — GET /api/users/{id}/dashboard — nada

4. SORPRESAS

- 4.1 Crear Sorpresa — POST /api/surprises — JSON {creator_id, genius_id, title, description, price, deadline, skill_id, size, is_urgent}
- 4.2 Actualizar Sorpresa — PUT /api/surprises/{id} — JSON según estado (open o in_progress)
- 4.3 Listar Sorpresas — GET /api/surprises — nada
- 4.4 Obtener Sorpresa — GET /api/surprises/{id} — nada
- 4.5 Eliminar Sorpresa — DELETE /api/surprises/{id} — nada
- 4.6 Iniciar Sorpresa — POST /api/surprises/{id}/start — nada (Header: Bearer token)
- 4.7 Entregar Sorpresa — POST /api/surprises/{id}/deliver — nada (Header: Bearer token)
- 4.8 Completar Sorpresa — POST /api/surprises/{id}/complete — nada (Header: Bearer token)
- 4.9 Cancelar Sorpresa — POST /api/surprises/{id}/cancel — JSON {reason_key, reason_text}
- 4.10 Sorpresas Creadas por Usuario — GET /api/users/{id}/surprises-created — nada
- 4.11 Sorpresas Realizadas por Genius — GET /api/users/{id}/surprises-genius — nada

5. DISPUTAS

- 5.1 Abrir Disputa — POST /api/surprises/{id}/dispute — JSON {reason} (Header: Bearer token)
- 5.2 Ver Disputa — GET /api/disputes/{id} — nada (Header: Bearer token)
- 5.3 Mis Disputas — GET /api/my/disputes — nada (Header: Bearer token)
- 5.4 Disputas como Creador — GET /api/users/{id}/disputes/creator — nada
- 5.5 Disputas como Genius — GET /api/users/{id}/disputes/genius — nada

6. CONVERSACIONES Y MENSAJES

- 6.1 Listar Conversaciones — GET /api/conversations — nada (Header: Bearer token)
- 6.2 Crear/Obtener Conversación — POST /api/conversations — JSON {creator_id, genius_id} (Header: Bearer token)
- 6.3 Eliminar Conversación — DELETE /api/conversations/{conversation} — nada (Header: Bearer token)
- 6.4 Listar Mensajes — GET /api/conversations/{conversationId}/messages — nada (Header: Bearer token)
- 6.5 Enviar Mensaje — POST /api/messages — JSON {conversation_id, message, file(opcional)} (Header: Bearer token)
- 6.6 Enviar Mensaje desde Sorpresa — POST /api/surprises/{surpriseId}/messages — JSON {message, file(opcional)} (Header: Bearer token)
- 6.7 Descargar Archivo de Mensaje — GET /api/messages/file/{id} — nada (Header: Bearer token)

7. OFERTAS Y NEGOCIACIÓN

- 7.1 Enviar Oferta — POST /api/surprises/{surpriseId}/offers — JSON {price, message, eta_hours}
- 7.2 Listar Ofertas — GET /api/surprises/{surpriseId}/offers — nada
- 7.3 Enviar Contraoferta — POST /api/offers/{offerId}/counter — JSON {price, message, eta_hours}
- 7.4 Aceptar Oferta — POST /api/offers/{offerId}/accept — nada

8. ARCHIVOS

- 8.1 Subir Archivo — POST /api/surprises/{id}/files — JSON {file} (Header: Bearer token)
- 8.2 Listar Archivos — GET /api/surprises/{id}/files — nada
- 8.3 Descargar Archivo — GET /api/files/{id}/download — nada

9. NOTIFICACIONES

9.1 Listar Notificaciones — GET /api/users/{id}/notifications — nada (Header: Bearer token)

9.2 Marcar Notificación Leída — POST /api/notifications/{id}/read — nada (Header: Bearer token)

9.3 Marcar Todas Leídas — POST /api/users/{id}/notifications/read-all — nada (Header: Bearer token)

10. REVIEWS

10.1 Crear Review — POST /api/surprises/{id}/review — JSON {rating, comment} (Header: Bearer token)

10.2 Listar Reviews — GET /api/users/{id}/reviews — nada

10.3 Obtener Rating — GET /api/users/{id}/rating — nada

11. SKILLS

11.1 Listar Skills — GET /api/skills — nada

11.2 Skills de Usuario — GET /api/users/{id}/skills — nada

11.3 Actualizar Skills Propuestas — POST /api/users/{id}/proposed-skills — JSON {skills: []}

11.4 Top por Skill — GET /api/skills/{skillId}/top — nada

11.5 Ranking por Skill — GET /api/skills/{skillId}/ranking — nada

11.6 Progreso en Skill — GET /api/users/{userId}/skills/{skillId}/progress — nada

11.7 Progreso Total — GET /api/users/{userId}/skills/progress — nada

11.8 Historial por Skill — GET /api/users/{userId}/skills/{skillId}/history — nada

11.9 Nivel Global — GET /api/users/{userId}/level — nada

11.10 Dashboard Skills — GET /api/users/{userId}/skills/dashboard — nada

11.11 Sugerencias Genius — GET /api/skills/{skillId}/genius-suggestions — nada

12. ADMIN

12.1 Dashboard Admin — GET /api/admin/dashboard — nada (Header: Bearer token admin)

12.2 Listar Disputas Admin — GET /api/admin/disputes — query opcional {status, winner, creator_id, genius_id, surprise_id, date_from, date_to, search}

12.3 Resolver Disputa — POST /api/admin/disputes/{id}/resolve — JSON {winner, resolution}

12.4 Listar Usuarios Admin — GET /api/admin/users — nada

12.5 Banear Usuario — POST /api/admin/users/{id}/ban — nada

12.6 Desbanear Usuario — POST /api/admin/users/{id}/unban — nada

12.7 Listar Sorpresas Admin — GET /api/admin/surprises — nada

12.8 Cancelación Forzada — POST /api/admin/surprises/{id}/force-cancel — nada